



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА имени К. А. ТИМИРЯЗЕВА»
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления
Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии
Направленность:
Большие данные и машинное обучение

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка веб-приложения «Task Manager»
для эффективного управления личными задачами».

Выполнил:
Студент 3 курса группы Д-Э317
Максимкин Захар Ильич

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО подпись

учёная степень, учёное звание, ФИО подпись

Оценка _____

Дата защиты _____

Москва 2026

Содержание

Введение	3
Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ - ПРИЛОЖЕНИЙ ДЛЯ УПРАВЛЕНИЯ ЗАДАЧАМИ	5
1.1. Анализ существующих решений для управления задачами . . .	5
1.2. Обзор технологий для разработки веб-приложений	6
1.3. Архитектурные подходы в веб-разработке	8
Глава 2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ВЕБ - ПРИЛОЖЕНИЯ «TASK MANAGER»	11
2.1. Функциональные и нефункциональные требования	11
2.2. Архитектура приложения	12
2.3. Структура проекта	13
2.4. Реализация фронтенд-части	15
2.5. Реализация бэкенд-части и развертывание	22
Заключение	23
Список использованных источников	26

Введение

В современных условиях цифровизации и ускоренного темпа жизни эффективное управление личными задачами становится критически важным навыком для повышения продуктивности как в профессиональной, так и в личной сфере. Традиционные методы планирования, такие как бумажные записные книжки или календари, уступают место цифровым инструментам, которые позволяют автоматизировать процесс управления задачами, обеспечить доступ к данным с любых устройств и интегрировать их с другими сервисами. Веб-приложения для управления задачами занимают особое место среди таких инструментов благодаря своей доступности, кроссплатформенности и возможности работы в реальном времени.

На рисунке 1 представлен интерфейс календарного вида приложения «Task Manager», где пользователь может видеть задачи на определенные даты, что демонстрирует удобство использования такого инструмента для планирования личных дел. Этот интерфейс позволяет пользователям быстро оценивать распределение задач по времени, что особенно важно для людей с плотным графиком.

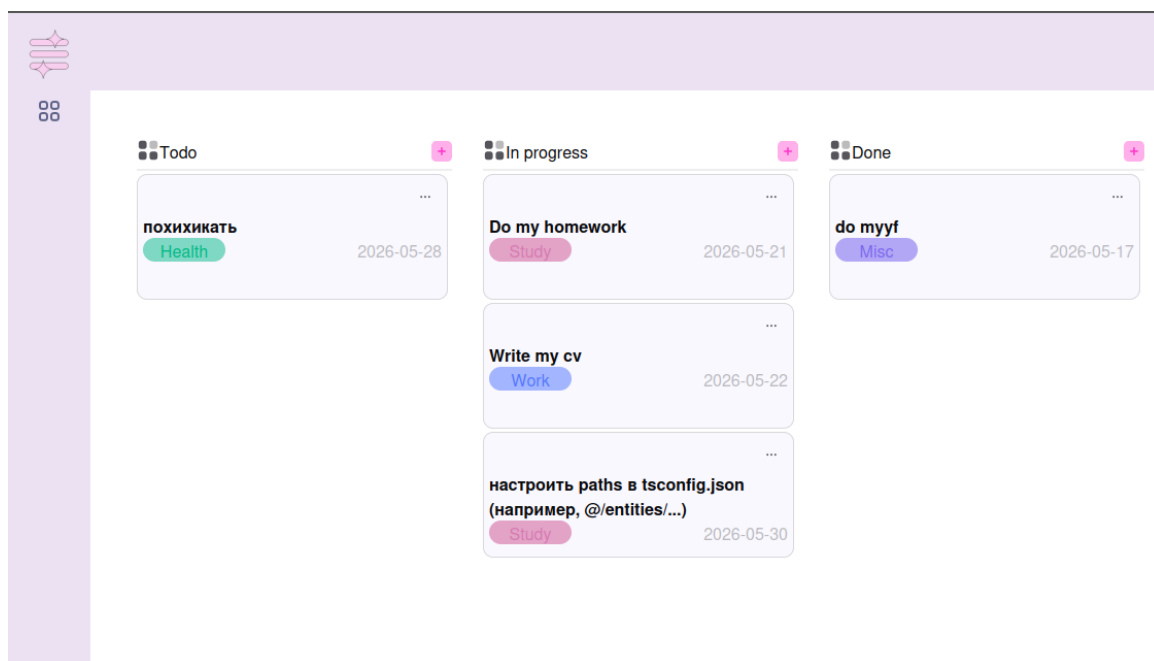


Рис. 1. Интерфейс календарного вида приложения «Task Manager»

Цифровые решения в этой области развиваются стремительными темпами.

От простых списков задач до сложных систем с поддержкой командной работы, интеграцией с календарями и уведомлениями. Однако большинство существующих решений либо перегружены избыточным функционалом, либо не учитывают индивидуальные потребности пользователей. Это определяет актуальность разработки гибкого, минималистичного и в то же время функционального веб-приложения для управления задачами, которое сочетает в себе простоту использования и необходимый набор функций.

Целью данной курсовой работы является проектирование и разработка веб-приложения «Task Manager» для эффективного управления личными задачами, с акцентом на реализацию фронтенд-части с использованием современных технологий и архитектурных подходов. Для достижения поставленной цели были сформулированы задачи по анализу существующих решений для управления задачами, изучению теоретических основ веб-разработки, проектированию архитектуры приложения, реализации фронтенд-части с использованием современных библиотек и фреймворков, а также обеспечению функциональности CRUD и механизма Drag-and-Drop для перетаскивания задач между статусами.

Научная новизна работы заключается в применении архитектурного подхода Feature-Sliced Design для структурирования веб-приложения, что обеспечивает высокую модульность, масштабируемость и удобство поддержки кода. Кроме того, в работе интегрированы современные технологии, такие как React 19, TypeScript, Zustand и TanStack Query, для создания эффективного и отказоустойчивого фронтенда. Особое внимание уделено реализации механизма Drag-and-Drop с использованием библиотеки @dnd-kit, что позволяет пользователям интуитивно управлять статусами задач, перетаскивая их между колонками "Todo", "In Progress" и "Done".

Практическая значимость разработанного веб-приложения «Task Manager» заключается в том, что оно может быть использовано как инструмент для личного управления задачами, что повышает продуктивность пользователей за счет автоматизации рутинных операций. Приложение служит основой для дальнейшего развития и добавления новых функций, таких как командная работа, интеграция с календарями и уведомлениями. Также оно может служить примером реализации современного веб-приложения с использованием актуальных технологий и архитектурных подходов для обучающих целей, демонстрируя студентам, как правильно структурировать и разрабатывать сложные веб-приложения.

Структура работы включает введение, две главы, заключение и список использованных источников. В первой главе рассмотрены теоретические основы разработки веб-приложений для управления задачами, включая анализ существующих решений, обзор технологий и архитектурных подходов. Вторая глава посвящена проектированию и реализации веб-приложения «Task Manager», включая функциональные требования, архитектуру, реализацию фронтенд- и бэкенд-частей, а также развертывание приложения.

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ - ПРИЛОЖЕНИЙ ДЛЯ УПРАВЛЕНИЯ ЗАДАЧАМИ

1.1. Анализ существующих решений для управления задачами

Современный рынок предлагает множество инструментов для управления задачами, которые можно классифицировать по функциональности, типу платформы и целевой аудитории. К числу наиболее популярных решений относятся Todoist, Trello, Notion, Microsoft To Do, Asana, ClickUp и множество других. Каждое из них имеет свои особенности, достоинства и недостатки, которые важно учитывать при выборе подходящего инструмента для конкретных задач.

Todoist представляет собой инструмент для работы с личными задачами, доступный на веб-платформах и мобильных устройствах. Его основные преимущества заключаются в простом и интуитивно понятном интерфейсе, а также кроссплатформенности, что позволяет пользователям работать с задачами на различных устройствах. Приложение поддерживает создание проектов, установку дедлайнов, добавление меток и приоритетов, а также интеграцию с другими сервисами, такими как Google Calendar и Dropbox. Однако бесплатная версия Todoist имеет ограниченный функционал: пользователи могут создать только 5 проектов, а также не имеют доступа к таким функциям, как напоминания и комментарии к задачам. Это может стать серьезным препятствием для пользователей, которым требуются расширенные возможности управления задачами.

Notion — универсальный инструмент, доступный на веб-платформах, мобильных устройствах и десктопе. Он подходит как для личного, так и для командного использования, и его основное преимущество заключается в универсальности и возможности кастомизации под любые задачи. Notion сочетает в себе функции текстовых редакторов, баз данных, вики и инструментов для управления задачами. Пользователи могут создавать страницы с различными типами контента: текст, таблицы, базы данных, календари, доски kanban и многое другое. Однако высокая сложность настройки Notion требует значительных временных затрат, что может оттолкнуть пользователей, которые хотят быстро начать работу. Для эффективного использования Notion необходимо изучить множество функций и возможностей платформы, что может занять значительное время.

Microsoft To Do — инструмент для управления личными задачами, доступный на веб-платформах и мобильных устройствах. Его основное преимущество — тесная интеграция с экосистемой Office 365, что делает его удобным выбо-

ром для пользователей, уже работающих с продуктами Microsoft. Приложение поддерживает создание списков задач, установку напоминаний и синхронизацию между устройствами. В то же время функционал приложения ограничен по сравнению с другими решениями, что может не удовлетворить потребности более требовательных пользователей. Отсутствие таких функций, как добавление вложений, комментариев или интеграция с другими сервисами, кроме продуктов Microsoft, может стать серьезным ограничением для некоторых пользователей.

Анализ существующих решений показал, что большинству из них не хватает баланса между простотой и функциональностью. Например, Todoist и Microsoft To Do подходят для личного использования, но не поддерживают командную работу и имеют ограниченный функционал. Trello и Notion более гибкие, но их настройка требует значительных временных затрат и может быть сложной для новичков. Asana и ClickUp предлагают широкий набор функций, но их сложность и стоимость могут быть избыточными для пользователей, которым требуется простой инструмент для управления личными задачами. Это подтверждает актуальность разработки минималистичного, но функционального веб-приложения для управления личными задачами, которое сочетает в себе простоту использования и необходимый набор функций для эффективной работы.

1.2. Обзор технологий для разработки веб-приложений

Современная веб-разработка включает широкий спектр технологий, которые можно условно разделить на фронтенд и бэкенд. Каждая из этих областей имеет свои особенности, инструменты и подходы, которые важно учитывать при разработке веб-приложений.

Фронтенд отвечает за интерфейс пользователя и его взаимодействие с приложением. Ключевые технологии для фронтенд-разработки включают React 19, TypeScript, Vite, Zustand, TanStack Query, React Hook Form, Chakra UI, React Router и @dnd-kit. Каждая из этих технологий выполняет свою роль в создании современного, эффективного и удобного пользовательского интерфейса.

React 19 — это библиотека для создания пользовательских интерфейсов на основе компонентов. Она была разработана компанией Facebook и в настоящее время является одной из самых популярных библиотек для разработки веб-приложений. React позволяет разработчикам создавать сложные и динамичные интерфейсы, которые легко поддерживать и масштабировать. Основные особенности React включают виртуальный DOM, который обеспечивает высокую производительность за счет минимального обновления только тех частей интерфейса, которые изменились, а также компонентный подход, который позволяет разделять интерфейс на независимые и переиспользуемые компоненты.

TypeScript представляет собой типизированный JavaScript, который был разработан компанией Microsoft. Он повышает надежность кода за счет статиче-

ской типизации, что позволяет выявлять ошибки на этапе компиляции, а не во время выполнения программы. TypeScript добавляет систему типов к JavaScript, что позволяет разработчикам определять типы переменных, параметров функций и возвращаемых значений, что значительно облегчает поддержку и рефакторинг кода. Кроме того, TypeScript поддерживает все функции JavaScript, что делает его совместимым с существующими проектами на JavaScript.

TanStack Query (ранее React Query) — это библиотека для работы с серверным состоянием, которая предоставляет механизмы кэширования, синхронизации данных и управления запросами. Она была разработана для решения проблемы сложности управления серверным состоянием в React-приложениях. TanStack Query позволяет разработчикам легко кэшировать данные, автоматически обновлять кэш при изменении данных на сервере, а также обрабатывать ошибки и состояния загрузки. Это значительно упрощает работу с асинхронными данными и улучшает пользовательский опыт за счет уменьшения количества запросов к серверу.

React Hook Form — это библиотека для управления формами и валидации в React-приложениях. Она была разработана для решения проблемы сложности работы с формами в React, особенно при использовании контролируемых компонентов. React Hook Form предлагает простой и эффективный способ управления состоянием форм, валидации данных и обработки отправки форм. Основные особенности библиотеки включают минимальное количество повторного рендеринга, что обеспечивает высокую производительность, а также удобный API для работы с формами любой сложности.

Chakra UI — это библиотека компонентов для стилизации интерфейса, которая была разработана для ускорения процесса разработки веб-приложений. Она предлагает набор готовых компонентов, которые можно легко интегрировать в проект и кастомизировать под свои нужды. Chakra UI основана на системе дизайна и предоставляет компоненты для создания доступных и адаптивных интерфейсов. Основные особенности библиотеки включают модульность, что позволяет использовать только те компоненты, которые необходимы, а также поддерживаемость тем, что позволяет легко изменять внешний вид приложения.

@dnd-kit — это библиотека для реализации механизма Drag-and-Drop в React-приложениях. Она была разработана как современная и гибкая альтернатива существующим решениям, таким как react-dnd. @dnd-kit предоставляет низкоуровневые примитивы для создания кастомных решений перетаскивания, что позволяет разработчикам реализовывать сложные сценарии Drag-and-Drop с минимальными затратами на настройку и поддержку. Основные особенности библиотеки включают высокую производительность, поддержку сенсорных устройств, а также возможность интеграции с другими библиотеками и фреймворками.

Бэкенд отвечает за обработку данных и бизнес-логику приложения. Ключевые технологии для бэкенд-разработки включают Node.js, Express, SQLite и

better-sqlite3. Каждая из этих технологий играет важную роль в создании надежной и эффективной серверной части приложения.

Node.js — это среда выполнения JavaScript на сервере, которая была разработана для решения проблемы невозможности выполнения JavaScript-кода вне браузера. Node.js позволяет использовать JavaScript для серверной разработки, что обеспечивает единство языка программирования как на клиенте, так и на сервере. Основные особенности Node.js включают асинхронную модель выполнения, которая обеспечивает высокую производительность при обработке большого количества одновременно подключенных клиентов, а также обширную экосистему модулей, которые можно использовать для решения различных задач.

SQLite — это легкая встраиваемая база данных, которая не требует отдельного сервера и идеально подходит для небольших и средних проектов. Она была разработана для решения проблемы сложности настройки и управления традиционными системами управления базами данных. SQLite предоставляет полноценную базу данных в одном файле, что делает ее удобной для использования в приложениях, где не требуется высокая производительность и масштабируемость. Основные особенности SQLite включают поддержку большинства функций SQL, что позволяет использовать ее для решения широкого круга задач, а также высокую надежность и устойчивость к сбоям.

better-sqlite3 — это библиотека для работы с SQLite в Node.js, которая была разработана для решения проблемы низкой производительности при использовании стандартных драйверов SQLite. Она предоставляет синхронный API для выполнения запросов к базе данных, что обеспечивает высокую скорость работы и удобство использования. Основные особенности better-sqlite3 включают поддержку подготовленных запросов, что позволяет повысить производительность при выполнении повторяющихся запросов, а также возможность работы с большими объемами данных.

1.3. Архитектурные подходы в веб-разработке

Современные веб-приложения требуют четкой архитектуры для обеспечения масштабируемости, поддерживаемости и удобства разработки. Существует множество архитектурных подходов, каждый из которых имеет свои особенности, преимущества и недостатки. Выбор подходящей архитектуры зависит от специфики проекта, его масштаба, команды разработчиков и других факторов.

Одним из актуальных архитектурных подходов является Feature-Sliced Design (FSD), который фокусируется на разделении приложения на независимые функциональные модули, так называемые фичи. Каждая фича включает в себя все необходимые компоненты: пользовательский интерфейс, бизнес-логику, стили, тесты и другие связанные элементы. Такой подход обеспечивает высокую модульность и переиспользуемость кода, что делает приложение более удобным

для поддержки и масштабирования.

Основные принципы FSD включают изоляцию фич друг от друга, что позволяет разрабатывать, тестировать и поддерживать их независимо. Это значительно упрощает процесс разработки, особенно в командах, где разные разработчики могут работать над разными фичами одновременно, не мешая друг другу. Кроме того, FSD способствует лучшему пониманию структуры приложения, так как каждая фича представляет собой законченный функциональный блок, который легко анализировать и модифицировать.

На рисунке 2 представлен интерфейс доски задач, реализованный с использованием подхода Feature-Sliced Design. Этот интерфейс демонстрирует, как задачи могут быть организованы в колонки в зависимости от их статуса, что позволяет пользователям легко отслеживать прогресс выполнения задач. Каждая колонка представляет собой отдельную фичу, которая может быть разработана и поддержана независимо от других фич.

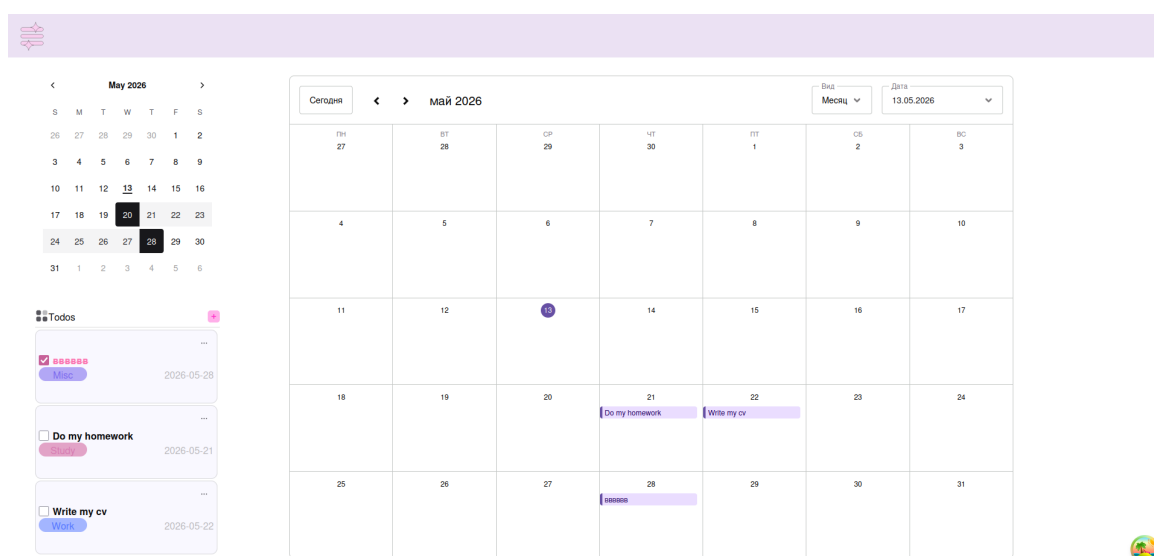


Рис. 1.1. Интерфейс доски задач с механизмом Drag-and-Drop

Помимо FSD, существуют и другие архитектурные подходы, которые могут быть использованы при разработке веб-приложений. Например, MVC (Model-View-Controller) — это классический архитектурный паттерн, который разделяет приложение на три основных компонента: модель, представление и контроллер. Модель отвечает за работу с данными, представление — за отображение данных пользователю, а контроллер — за обработку действий пользователя и обновление модели и представления. Этот подход обеспечивает четкое разделение ответственности между компонентами, что упрощает поддержку и тестирование приложения.

Еще одним популярным архитектурным подходом является микросервисная архитектура, которая предполагает разделение приложения на небольшие независимые сервисы, каждый из которых выполняет свою функцию. Каждый

микросервис может быть разработан, развернут и масштабирован независимо от других сервисов, что обеспечивает высокую гибкость и масштабируемость приложения. Однако микросервисная архитектура также имеет свои недостатки, такие как сложность управления и координации между сервисами, а также увеличенные затраты на инфраструктуру и развертывание.

Выбор подходящей архитектуры зависит от множества факторов, включая масштаб проекта, требования к производительности и масштабируемости, состав команды разработчиков и их опыт работы с теми или иными подходами. Для небольших и средних проектов, таких как приложение «Task Manager», подход FSD может быть оптимальным выбором, так как он обеспечивает хороший баланс между модульностью, поддерживаемостью и простотой разработки.

Глава 2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ВЕБ - ПРИЛОЖЕНИЯ «TASK MANAGER»

2.1. Функциональные и нефункциональные требования

Функциональные требования к приложению «Task Manager» включают в себя широкий набор возможностей, которые позволяют пользователям эффективно управлять своими задачами. В первую очередь, приложение должно предоставлять возможность авторизации пользователей, что позволяет обеспечивать безопасность данных и персонализацию опыта использования. Регистрация и вход в систему должны быть простыми и интуитивно понятными, чтобы пользователи могли быстро начать работу с приложением.

Управление задачами является ключевой функцией приложения и включает в себя поддержку операций CRUD (Create, Read, Update, Delete). Пользователи должны иметь возможность создавать новые задачи, просматривать список существующих задач, редактировать задачи и удалять их при необходимости. Каждая задача должна иметь такие атрибуты, как заголовок, описание, статус, дата создания и дата обновления. Статус задачи может принимать одно из нескольких значений, таких как "Todo", "In Progress" или "Done", что позволяет пользователям отслеживать прогресс выполнения задач.

Помимо базовых операций CRUD, приложение должно поддерживать перетаскивание задач между статусами с использованием механизма Drag-and-Drop. Это позволяет пользователям интуитивно изменять статус задач, перетаскивая их между колонками, что значительно упрощает процесс управления задачами и делает его более наглядным. Механизм Drag-and-Drop должен быть реализован таким образом, чтобы обеспечивать плавное и естественное перетаскивание задач, а также визуальную обратную связь для пользователя.

Кроме того, приложение должно поддерживать фильтрацию и сортировку задач по различным критериям. Пользователи должны иметь возможность фильтровать задачи по статусу, дате создания или другим атрибутам, а также сортировать их по дате создания, дате обновления или алфавиту. Это позволяет пользователям быстро находить нужные задачи и организовывать их в удобном порядке.

Нефункциональные требования к приложению включают высокую производительность, надёжность, безопасность данных пользователей, удобство использования и кроссплатформенность. Приложение должно быстро отзываться на действия пользователя, даже при работе с большим количеством задач. Это

требует оптимизации кода и использования эффективных алгоритмов и структур данных.

Надёжность приложения является критически важным аспектом, так как пользователи будут доверять ему свои данные и ожидать, что приложение будет работать стабильно и без сбоев. Для обеспечения надёжности необходимо предусмотреть обработку ошибок, валидацию данных, а также механизмы восстановления после сбоев. Например, при ошибке подключения к серверу приложение должно уведомлять пользователя и предоставлять возможность повторной попытки или работы в оффлайн-режиме.

Безопасность данных пользователей является еще одним важным аспектом, который необходимо учитывать при разработке приложения. Все данные пользователей, включая личную информацию и задачи, должны быть защищены от несанкционированного доступа. Для этого необходимо использовать современные методы аутентификации и авторизации, такие как JWT (JSON Web Tokens), а также обеспечивать безопасное хранение данных в базе данных.

Удобство использования приложения также имеет критически важное значение для его успеха. Интерфейс приложения должен быть интуитивно понятным и простым в использовании, чтобы пользователи могли быстро освоить его и начать эффективную работу. Для этого необходимо уделять внимание дизайну интерфейса, удобству навигации, а также предоставлению понятных сообщений и обратной связи для пользователя.

Кроссплатформенность приложения означает, что оно должно работать на всех современных браузерах и устройствах, включая десктопы, планшеты и смартфоны. Это требует использования адаптивного дизайна, который автоматически подстраивается под размеры экрана устройства, а также тестирования приложения на различных платформах и браузерах.

2.2. Архитектура приложения

Приложение «Task Manager» построено на основе клиент-серверной архитектуры, которая предполагает разделение приложения на две основные части: клиентскую (фронтенд) и серверную (бэкенд). Клиентская часть отвечает за интерфейс пользователя и его взаимодействие с приложением, а серверная часть — за обработку данных, бизнес-логику и работу с базой данных.

Фронтенд-часть приложения разработана с использованием современных технологий и библиотек, которые обеспечивают высокую производительность, удобство разработки и качество конечного продукта. В качестве основной библиотеки для создания пользовательского интерфейса используется React 19, которая позволяет разработчикам создавать сложные и динамичные интерфейсы на основе компонентов. Для повышения надежности кода и упрощения его поддержки используется TypeScript, который добавляет систему типов к JavaScript.

Для сборки фронтенд-части приложения используется Vite, который обеспечивает высокую скорость разработки за счет быстрой перезагрузки модулей и оптимизированного процесса сборки. Управление состоянием приложения реализовано с использованием библиотеки Zustand, которая предоставляет простой и эффективный способ управления глобальным состоянием.

Работа с серверным состоянием и данными реализована с использованием библиотеки TanStack Query, которая предоставляет механизмы кэширования, синхронизации данных и управления запросами. Управление формами и валидация данных реализованы с использованием библиотеки React Hook Form. Для стилизации интерфейса используется библиотека Chakra UI, которая предоставляет набор готовых компонентов для создания красивых и доступных интерфейсов. Маршрутизация в приложении реализована с использованием библиотеки React Router.

Бэкенд-часть приложения разработана с использованием Node.js и фреймворка Express, которые обеспечивают высокую производительность и гибкость при разработке серверной части. Для работы с базой данных используется SQLite с библиотекой better-sqlite3, которая предоставляет синхронный API для выполнения запросов к базе данных.

Структура проекта организована в соответствии с подходом Feature-Sliced Design, который предполагает разделение приложения на независимые функциональные модули. Фронтенд-часть проекта включает директории для фич, общих компонентов и утилит, страниц приложения и конфигурации. Каждая фича представляет собой законченный функциональный блок, который включает в себя все необходимые компоненты: пользовательский интерфейс, бизнес-логику, стили и тесты.

Бэкенд-часть проекта включает директории для маршрутов API, контроллеров, моделей данных и конфигурации базы данных. Маршруты API предоставляют эндпоинты для работы с задачами и пользователями, контроллеры обрабатывают запросы и взаимодействуют с моделями данных, а модели данных определяют структуру данных, хранящихся в базе данных.

Такой подход к организации проекта обеспечивает четкое разделение ответственности между различными частями приложения, что упрощает процесс разработки, тестирования и поддержки. Кроме того, он позволяет легко масштабировать приложение по мере роста его функциональности и сложности.

2.3. Структура проекта

Структура проекта «Task Manager» организована в соответствии с подходом Feature-Sliced Design, что обеспечивает высокую модульность и удобство поддержки кода. В этом разделе мы подробно рассмотрим структуру как фронтенд-, так и бэкенд-части проекта.

Фронтенд-часть проекта включает следующие основные директории и файлы:

Директория `/app` содержит весь исходный код клиентской части приложения. Внутри этой директории расположены поддиректории, каждая из которых отвечает за определенный аспект приложения.

Директория `/features` содержит все функциональные модули (фичи) приложения. Каждая фича представляет собой независимый модуль, который включает в себя все необходимые компоненты для реализации определенной функциональности. Например, фича `/auth` отвечает за авторизацию пользователей и включает в себя компоненты интерфейса, бизнес-логику, стили и тесты, связанные с авторизацией. Такая организация позволяет разрабатывать и поддерживать каждую фичу независимо от других, что значительно упрощает процесс разработки, особенно в командах.

Директория `/shared` содержит общие компоненты, утилиты и стили, которые могут быть использованы в различных фичах. Например, здесь могут находиться общие компоненты интерфейса, такие как кнопки, формы ввода, модальные окна, а также утилитарные функции, которые используются в разных частях приложения. Это позволяет избежать дублирования кода и обеспечивает его переиспользуемость.

Директория `/pages` содержит компоненты страниц приложения, которые представляют собой верхнеуровневые компоненты, отвечающие за отображение различных страниц приложения. Каждая страница может использовать компоненты из различных фич и общих модулей для создания законченного пользовательского интерфейса.

Директория `/app` содержит конфигурацию приложения, включая настройки маршрутизации, глобальные стили, а также основные компоненты приложения, такие как корневой компонент `App`, который отвечает за общую структуру приложения.

Бэкенд-часть проекта включает следующие основные директории и файлы:

Директория `/server` содержит весь исходный код серверной части приложения. Внутри этой директории расположены поддиректории, каждая из которых отвечает за определенный аспект серверной части.

Директория `/routes` содержит файлы маршрутизации, которые определяют эндпоинты API и связывают их с соответствующими контроллерами. Каждый файл маршрутизации отвечает за определенную область функциональности, например, маршруты для работы с задачами или маршруты для авторизации пользователей.

Директория `/controllers` содержит контроллеры, которые обрабатывают запросы от клиентской части и взаимодействуют с моделями данных. Каждый контроллер отвечает за обработку запросов к определенному эндпоинту API и содержит бизнес-логику, связанную с этим эндпоинтом.

Директория `/models` содержит модели данных, которые определяют струк-

туру данных, хранящихся в базе данных, а также методы для работы с этими данными. Каждая модель отвечает за определенную сущность в базе данных, например, пользователей или задачи, и предоставляет методы для создания, чтения, обновления и удаления записей.

Директория `/db` содержит конфигурацию базы данных, включая настройки подключения к базе данных и миграции. В проекте используется SQLite с библиотекой `better-sqlite3`, что позволяет легко настраивать и использовать базу данных без необходимости установки и настройки отдельного сервера базы данных.

Файл `app.ts` является основным файлом серверной части приложения, который отвечает за настройку и запуск сервера. В этом файле происходит настройка `middleware`, подключение маршрутов, а также запуск сервера на определенном порту.

Файл `server.ts` отвечает за запуск сервера и обработку ошибок. В этом файле происходит запуск сервера и настройка обработчиков ошибок, что обеспечивает стабильную работу сервера даже в случае возникновения ошибок.

2.4. Реализация фронтенд-части

Настройка проекта фронтенд-части началась с создания проекта с использованием `Vite` и `TypeScript`. `Vite` был выбран в качестве инструмента сборки за его способность обеспечивать высокую скорость разработки благодаря быстрой перезагрузке модулей и оптимизированному процессу сборки. `TypeScript` был выбран за его способность повышать надежность кода за счет статической типизации, что позволяет выявлять ошибки на этапе компиляции, а не во время выполнения программы.

Создание проекта было выполнено с использованием команды, представленной ниже. Эта команда создает новый проект с использованием шаблона `React + TypeScript` и настраивает его для работы с `Vite`.

```
1 npm create vite@latest task-manager -- --template react-ts
2 cd task-manager
```

Листинг 2.1. Создание проекта с использованием `Vite` и `TypeScript`

После создания проекта была выполнена установка необходимых зависимостей, которые включают библиотеки для управления состоянием, работы с серверным состоянием, управления формами, стилизации интерфейса и маршрутизации. Установка зависимостей была выполнена с использованием команды `npm install`, которая устанавливает все указанные пакеты и их зависимости.

```
1 npm install @tanstack/react-query @chakra-ui/react @emotion/react @emotion/
  styled framer-motion react-hook-form react-router-dom zustand @dnd-kit/core
  @dnd-kit/sortable @dnd-kit/utilities
```

Листинг 2.2. Установка зависимостей для фронтенд-части

Каждая из установленных библиотек играет важную роль в разработке фронтенд - части приложения. `@tanstack/react-query` используется для работы с серверным состоянием и кэширования данных. `@chakra-ui/react`, `@emotion/react` и `@emotion/styled` используются для стилизации интерфейса и создания красивых и доступных компонентов. `framer-motion` используется для добавления анимаций к компонентам интерфейса. `react-hook-form` используется для управления формами и валидации данных. `react-router-dom` используется для маршрутизации в приложении. `zustand` используется для управления глобальным состоянием приложения. `@dnd-kit/core`, `@dnd-kit/sortable` и `@dnd-kit/utilities` используются для реализации механизма Drag-and-Drop.

Управление состоянием в приложении реализовано с использованием библиотеки Zustand. Zustand был выбран за его минимализм и простоту использования, что делает его идеальным выбором для проектов средней сложности, где не требуется вся мощь и сложность таких решений, как Redux. На рисунке 3 представлен интерфейс формы входа в приложение, реализованный с использованием Chakra UI. Эта библиотека была выбрана за ее способность предоставлять набор готовых компонентов, которые упрощают создание красивых и доступных интерфейсов.



Рис. 2.1. Интерфейс формы входа в приложение

Пример создания store для управления состоянием авторизации представлен ниже. Этот store позволяет управлять состоянием авторизации пользователя, включая хранение информации о пользователе и его статусе авторизации. Store содержит состояние пользователя и флажок, указывающий, авторизован ли пользователь. Также он предоставляет методы для входа и выхода из системы, которые обновляют состояние store.

```
1 import { create } from 'zustand';
2
3 interface AuthState {
4   user: { id: number; username: string } | null;
5   isAuthenticated: boolean;
6   login: (user: { id: number; username: string }) => void;
```

```

7   logout: () => void;
8 }
9
10 export const useAuthStore = create<AuthState>((set) => ({
11   user: null,
12   isAuthenticated: false,
13   login: (user) => set({ user, isAuthenticated: true }),
14   logout: () => set({ user: null, isAuthenticated: false }),
15 }));

```

Листинг 2.3. Пример создания store для авторизации с использованием Zustand

Этот store используется в компонентах приложения для управления состоянием авторизации. Например, в компоненте формы входа можно использовать метод `login` для обновления состояния после успешной авторизации пользователя. Это позволяет легко отслеживать состояние авторизации пользователя в разных частях приложения и обновлять интерфейс в зависимости от этого состояния.

Работа с серверным состоянием реализована с использованием библиотеки TanStack Query. Эта библиотека была выбрана за ее способность предоставлять механизмы кэширования и синхронизации данных, что позволяет снизить нагрузку на сервер и улучшить пользовательский опыт. TanStack Query автоматически кэширует данные, полученные от сервера, и обновляет кэш при изменении данных, что позволяет уменьшить количество запросов к серверу и ускорить работу приложения.

Пример запроса для получения списка задач представлен ниже. Этот запрос использует хук `useQuery` из TanStack Query для получения данных о задачах с сервера. Хук принимает в качестве параметров ключ кэша и функцию для получения данных. Ключ кэша используется для идентификации данных в кэше, а функция получения данных выполняет фактический запрос к серверу.

```

1 import { useQuery } from '@tanstack/react-query';
2 import axios from 'axios';
3
4 const fetchTasks = async () => {
5   const response = await axios.get('/api/tasks');
6   return response.data;
7 };
8
9 export const useTasksQuery = () => {
10   return useQuery({
11     queryKey: ['tasks'],
12     queryFn: fetchTasks,
13   });
14 };

```

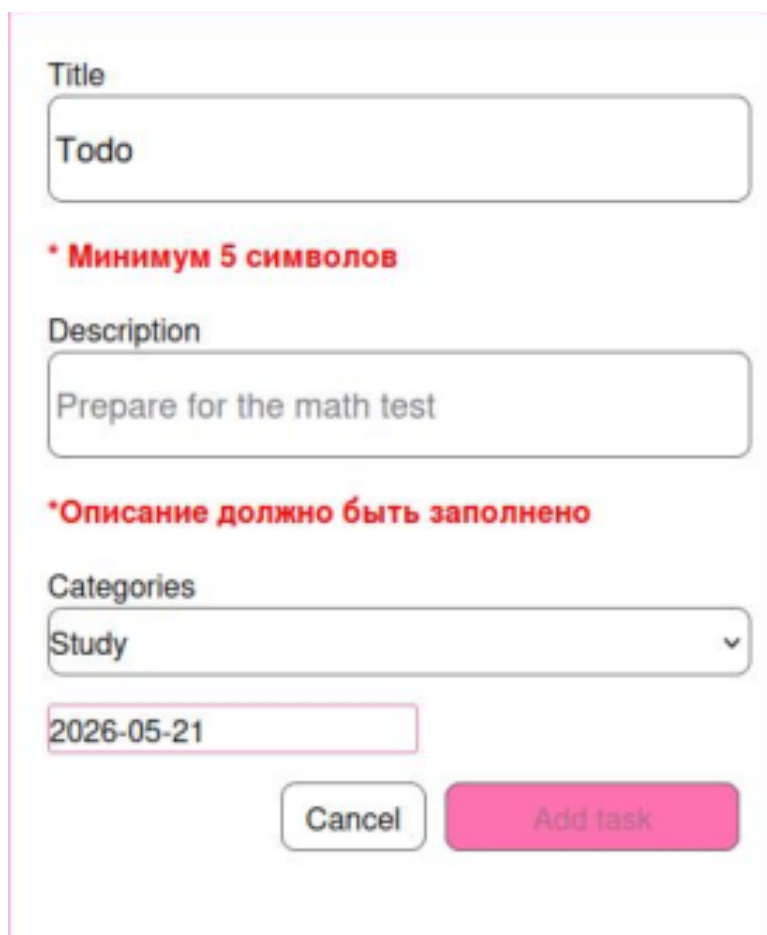
Листинг 2.4. Пример запроса для получения списка задач с использованием TanStack Query

Этот хук может быть использован в компонентах приложения для получения и отображения списка задач. TanStack Query автоматически обрабатывает

состояния загрузки и ошибок, что позволяет легко отображать соответствующие сообщения пользователю во время загрузки данных или при возникновении ошибок.

Управление формами реализовано с использованием библиотеки React Hook Form. Эта библиотека была выбрана за ее способность упрощать процесс работы с формами в React-приложениях, включая валидацию данных и управление состоянием форм. React Hook Form предоставляет удобный API для работы с формами любой сложности и обеспечивает высокую производительность за счет минимального количества повторных рендерингов.

На рисунке 4 показан интерфейс формы добавления задачи, который демонстрирует удобство использования приложения для создания новых задач. Форма включает поля для ввода заголовка и описания задачи, а также кнопку для отправки формы. Поля формы подключены к React Hook Form с использованием метода `register`, который обеспечивает автоматическое управление состоянием форм и валидацию данных.



The image shows a web form for adding a task. It has the following elements:

- Title:** A text input field with the value "Todo". Below it is a red error message: *** Минимум 5 символов**.
- Description:** A text input field with the value "Prepare for the math test". Below it is a red error message: ***Описание должно быть заполнено**.
- Categories:** A dropdown menu with "Study" selected.
- Date:** A date input field showing "2026-05-21".
- Buttons:** "Cancel" and "Add task" buttons at the bottom right.

Рис. 2.2. Интерфейс формы добавления задачи

Пример формы для добавления задачи представлен ниже. Эта форма использует хук `useForm` из React Hook Form для управления состоянием формы и валидации данных. Хук возвращает объект с методами и состоянием формы,

которые могут быть использованы для работы с формой.

```
1 import { useForm } from 'react-hook-form';
2 import { Button, Input, VStack } from '@chakra-ui/react';
3
4 interface TaskFormData {
5   title: string;
6   description: string;
7 }
8
9 const AddTaskForm = ({ onSubmit }: { onSubmit: (data: TaskFormData) => void })
10 => {
11   const { register, handleSubmit, formState: { errors } } = useForm<
12     TaskFormData>();
13
14   return (
15     <form onSubmit={handleSubmit(onSubmit)}>
16       <VStack spacing={4}>
17         <Input
18           placeholder="Title"
19           {...register('title', { required: 'Title is required' })}
20         />
21         {errors.title && <p style={{ color: 'red' }}>{errors.title.
22           message}</p>}
23         <Input
24           placeholder="Description"
25           {...register('description')}
26         />
27         <Button type="submit" colorScheme="teal">
28           Add Task
29         </Button>
30       </VStack>
31     </form>
32   );
33 };
34
35 export default AddTaskForm;
```

Листинг 2.5. Пример формы для добавления задачи с использованием React Hook Form

В этом примере форма включает два поля ввода для заголовка и описания задачи. Поле заголовка имеет обязательную валидацию, что означает, что пользователь должен заполнить это поле перед отправкой формы. Если поле заголовка не заполнено, React Hook Form автоматически добавляет ошибку в объект `errors`, который может быть использован для отображения сообщения об ошибке пользователю. Поле описания не является обязательным и может быть оставлено пустым.

Реализация механизма Drag-and-Drop выполнена с использованием библиотеки `@dnd-kit`, которая предоставляет низкоуровневые примитивы для создания кастомных решений перетаскивания. Этот механизм позволяет пользователям интуитивно изменять статус задач, перетаскивая их между колонками "Todo", "In Progress" и "Done". Это значительно упрощает процесс управления задачами и делает его более наглядным и интерактивным.

Пример настройки Drag-and-Drop представлен ниже. В этом примере используется контекст DndContext из `@dnd-kit/core`, который предоставляет функциональность для обработки событий перетаскивания. Компонент TasksBoard использует этот контекст для управления состоянием задач и обработки событий перетаскивания.

```
1 import { useState } from 'react';
2 import {
3   DndContext,
4   closestCenter,
5   KeyboardSensor,
6   PointerSensor,
7   useSensor,
8   useSensors,
9   DragEndEvent,
10 } from '@dnd-kit/core';
11 import {
12   arrayMove,
13   SortableContext,
14   sortableKeyboardCoordinates,
15 } from '@dnd-kit/sortable';
16
17 interface Task {
18   id: string;
19   title: string;
20   status: 'Todo' | 'InProgress' | 'Done';
21 }
22
23 const TasksBoard = () => {
24   const [tasks, setTasks] = useState<Task[]>([
25     { id: '1', title: 'Task_1', status: 'Todo' },
26     { id: '2', title: 'Task_2', status: 'InProgress' },
27     { id: '3', title: 'Task_3', status: 'Done' },
28   ]);
29
30   const sensors = useSensors(
31     useSensor(PointerSensor),
32     useSensor(KeyboardSensor, {
33       coordinateGetter: sortableKeyboardCoordinates,
34     })
35   );
36
37   const handleDragEnd = (event: DragEndEvent) => {
38     const { active, over } = event;
39     if (active.id !== over?.id) {
40       setTasks((items) => {
41         const oldIndex = items.findIndex((item) => item.id === active.id);
42         const newIndex = items.findIndex((item) => item.id === over?.id);
43         return arrayMove(items, oldIndex, newIndex);
44       });
45     }
46   };
47
48   return (
49     <DndContext
50       sensors={sensors}
```

```

51         collisionDetection={closestCenter}
52         onDragEnd={handleDragEnd}
53     >
54         <SortableContext items={tasks}>
55             { /* Здесь будут компоненты задач, которые можно перетаскивать */ }
56         </SortableContext>
57     </DndContext>
58 );
59 };
60
61 export default TasksBoard;

```

Листинг 2.6. Пример настройки механизма Drag-and-Drop с использованием @dnd-kit

В этом примере компонент TasksBoard управляет состоянием списка задач и обрабатывает события перетаскивания. При перетаскивании задачи между колонками обновляется состояние списка задач, что приводит к перерисовке интерфейса и отображению задач в новых позициях. Это позволяет пользователям интуитивно изменять статус задач и управлять ими.

Стилизация интерфейса приложения реализована с использованием библиотеки Chakra UI, которая предоставляет набор готовых компонентов для создания красивых и доступных интерфейсов. Chakra UI основана на системе дизайна и предоставляет компоненты для создания адаптивных и доступных интерфейсов, которые хорошо выглядят на любых устройствах.

Пример использования компонентов Chakra UI представлен ниже. В этом примере создается простой интерфейс с заголовком и кнопкой, который демонстрирует базовые возможности библиотеки.

```

1 import { Box, Heading, Button, VStack } from '@chakra-ui/react';
2
3 const App = () => {
4     return (
5         <Box p={4}>
6             <VStack spacing={8}>
7                 <Heading as="h1" size="xl">
8                     Task Manager
9                 </Heading>
10                <Button colorScheme="teal" onClick={() => console.log('Button_
11                    clicked')}>
12                    Add Task
13                </Button>
14            </VStack>
15        </Box>
16    );
17 };
18 export default App;

```

Листинг 2.7. Пример использования компонентов Chakra UI для стилизации интерфейса

В этом примере используется несколько компонентов из Chakra UI: Box для создания контейнера, Heading для отображения заголовка, Button для создания кнопки и VStack для организации компонентов в вертикальный стек с отступами между ними. Эти компоненты могут быть легко кастомизированы с использованием пропсов, что позволяет создавать уникальные и красивые интерфейсы без необходимости писать большой объем CSS-кода.

Маршрутизация в приложении реализована с использованием библиотеки React Router, которая предоставляет инструменты для создания навигации в одностраничных приложениях. React Router позволяет разработчикам создавать сложные навигационные структуры с вложенными маршрутами, динамической загрузкой компонентов и обработкой ошибок навигации.

Пример настройки маршрутов представлен ниже. В этом примере создается маршрутизатор с использованием функции createBrowserRouter из React Router. Маршрутизатор включает маршруты для главной страницы приложения и страницы входа, что позволяет пользователям перемещаться между этими страницами без перезагрузки страницы.

```
1 import { createBrowserRouter } from 'react-router-dom';
2 import { LoginPage } from '@pages/login';
3 import { TasksPage } from '@pages/tasks';
4 import { ProtectedRoute } from '@components/ProtectedRoute';
5
6 export const router = createBrowserRouter([
7   {
8     path: '/',
9     element: <ProtectedRoute><TasksPage /></ProtectedRoute>,
10  },
11  {
12    path: '/login',
13    element: <LoginPage />,
14  },
15 ]);
```

Листинг 2.8. Пример настройки маршрутов с использованием React Router

В этом примере используется компонент ProtectedRoute, который проверяет, авторизован ли пользователь, и перенаправляет его на страницу входа, если он не авторизован. Это обеспечивает защиту маршрутов, которые требуют авторизации, и предотвращает доступ неавторизованных пользователей к защищенным страницам приложения.

2.5. Реализация бэкенд-части и развертывание

Бэкенд-часть приложения «Task Manager» разработана на основе Node.js и фреймворка Express, которые обеспечивают высокую производительность и гибкость при разработке серверной части. Express был выбран за его минимализм и простоту использования, что делает его идеальным выбором для проектов средней сложности, где не требуется вся мощь и сложность более крупных

фреймворков.

Структура бэкенд-части приложения включает директории для маршрутов API, контроллеров, моделей данных и конфигурации базы данных. Каждая из этих директорий отвечает за определенный аспект серверной части приложения и содержит файлы, связанные с этим аспектом.

Директория `/routes` содержит файлы маршрутизации, которые определяют эндпоинты API и связывают их с соответствующими контроллерами. Каждый файл маршрутизации отвечает за определенную область функциональности, например, маршруты для работы с задачами или маршруты для авторизации пользователей. В проекте «Task Manager» есть несколько основных файлов маршрутизации: `auth.ts` для маршрутов авторизации и `tasks.ts` для маршрутов работы с задачами.

Директория `/controllers` содержит контроллеры, которые обрабатывают запросы от клиентской части и взаимодействуют с моделями данных. Каждый контроллер отвечает за обработку запросов к определенному эндпоинту API и содержит бизнес-логику, связанную с этим эндпоинтом. В проекте есть контроллеры для авторизации пользователей, работы с задачами и других функций.

Директория `/models` содержит модели данных, которые определяют структуру данных, хранящихся в базе данных, а также методы для работы с этими данными. Каждая модель отвечает за определенную сущность в базе данных, например, пользователей или задачи, и предоставляет методы для создания, чтения, обновления и удаления записей.

Директория `/db` содержит конфигурацию базы данных, включая настройки подключения к базе данных и миграции. В проекте используется SQLite с библиотекой `better-sqlite3`, что позволяет легко настраивать и использовать базу данных без необходимости установки и настройки отдельного сервера базы данных.

Работа с Git и GitHub была организована для управления версиями кода и совместной работы в команде. Git — это распределенная система контроля версий, которая позволяет отслеживать изменения в коде, работать с различными версиями проекта и объединять изменения от разных разработчиков. GitHub — это платформа для хостинга Git-репозитория, которая предоставляет инструменты для совместной работы, такие как `pull request`, `Issues` и `Projects`.

Основные команды Git, которые использовались в процессе разработки приложения, включают `git clone` для клонирования репозитория, `git add` для добавления изменений в индекс, `git commit` для создания коммитов, `git push` для отправки изменений в удаленный репозиторий и `git pull` для получения изменений из удаленного репозитория. Эти команды позволяют эффективно управлять кодом и синхронизировать изменения между локальным и удаленным репозиториями.

Заключение

В ходе выполнения курсовой работы было разработано веб-приложение «Task Manager» для эффективного управления личными задачами. В процессе работы были достигнуты значительные результаты, включая проведение анализа существующих решений для управления задачами, изучение теоретических основ веб-разработки, проектирование архитектуры приложения, реализацию фронтенд- и бэкенд-частей, а также развертывание приложения.

Была разработана архитектура приложения на основе подхода Feature-Sliced Design, что обеспечило высокую модульность и удобство поддержки кода. Этот подход позволил четко разделить ответственность между различными частями приложения и упростил процесс разработки и тестирования. Реализация фронтенд-части включала использование современных технологий, таких как React 19, TypeScript, Zustand, TanStack Query, React Hook Form и Chakra UI, что позволило создать эффективный и отказоустойчивый интерфейс пользователя.

Особое внимание было уделено реализации механизма Drag-and-Drop с использованием библиотеки @dnd-kit, что позволяет пользователям интуитивно управлять статусами задач, перетаскивая их между колонками. Это значительно упрощает процесс управления задачами и делает его более наглядным и интерактивным. Была обеспечена функциональность CRUD для работы с задачами, что позволяет пользователям создавать, просматривать, редактировать и удалять задачи в удобном интерфейсе.

Реализована система авторизации пользователей с использованием JWT, что обеспечивает безопасность данных пользователей и защиту от несанкционированного доступа. Настроена работа с базой данных SQLite, что позволяет эффективно хранить и управлять данными о пользователях и их задачах. Разработаны эндпоинты REST API для взаимодействия между клиентской и серверной частями приложения, что обеспечивает стабильную и эффективную работу приложения.

Научная значимость работы заключается в применении архитектурного подхода Feature-Sliced Design для структурирования веб-приложения, что обеспечивает высокую модульность и удобство поддержки кода. Интеграция современных технологий для создания эффективного и отказоустойчивого фронтенда, а также реализация механизма Drag-and-Drop с использованием библиотеки @dnd-kit являются важными достижениями, которые могут быть полезны для других проектов и обучающих целей.

Практическая значимость разработанного веб-приложения «Task Manager»

заключается в том, что оно может быть использовано для личного управления задачами, что повышает продуктивность пользователей за счет автоматизации рутинных операций и предоставления удобного интерфейса для работы с задачами. Приложение служит основой для дальнейшего развития и добавления новых функций, таких как командная работа, интеграция с календарями, добавление системы уведомлений и многое другое.

В будущем возможно развитие приложения в нескольких направлениях. Во-первых, добавление поддержки командной работы позволит использовать приложение для управления задачами в командах, что расширит его функциональность и сделает более универсальным. Во-вторых, интеграция с календарями, такими как Google Calendar или Yandex Calendar, позволит пользователям синхронизировать свои задачи с календарными событиями, что улучшит планирование и организацию времени. В-третьих, добавление системы уведомлений, таких как email или push-уведомления, позволит напоминать пользователям о предстоящих задачах и дедлайнах, что повысит их продуктивность.

Кроме того, в будущем можно рассмотреть возможность добавления таких функций, как повторяющиеся задачи, календарный вид для просмотра задач, интеграция с другими сервисами и приложениями, а также добавление аналитики и отчетности для отслеживания производительности и эффективности работы пользователей. Эти функции могут сделать приложение еще более мощным и удобным инструментом для управления задачами.

Список использованных источников

1. Хавербеке, М. Выразительный JavaScript. Современное веб-программирование / М. Хавербеке. — 3-е изд. — СПб. : Питер, 2024. — 416 с.
2. Браун, И. Веб-разработка с применением Node и Express. Полноценное использование стека JavaScript / И. Браун. — 2-е изд. — СПб. : Питер, 2023. — 352 с.
3. Choi, D. Full-Stack React, TypeScript, and Node. Build scalable and cloud-ready web applications / D. Choi. — 2nd ed. — Birmingham : Packt Publishing, 2025. — 328 p.
4. Бэнкс, А. React: современные шаблоны для разработки приложений / А. Бэнкс, Е. Порселло ; пер. с англ. С. Черников. — 2-е изд. — СПб. : Питер, 2023. — 315 с.
5. Фримен, А. Основы TypeScript / А. Фримен. — 3-е изд. — СПб. : Питер, 2024. — 608 с.
6. Григгс, Б. Node.js Cookbook / Б. Григгс, М. Спиголон. — 5th ed. — Birmingham : Packt Publishing, 2024. — 456 p.
7. Официальная документация React : веб-сайт. — URL: <https://react.dev/> (дата обращения: 31.05.2026).
8. Официальная документация TypeScript : веб-сайт. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 31.05.2026).
9. Официальная документация Node.js : веб-сайт. — URL: <https://nodejs.org/en/docs/> (дата обращения: 31.05.2026).
10. Feature-Sliced Design. A New Architecture for Frontend Applications : веб-сайт. — URL: <https://feature-sliced.design/docs> (дата обращения: 31.05.2026).
11. Официальная документация SQLite : веб-сайт. — URL: <https://www.sqlite.org/docs.html> (дата обращения: 31.05.2026).
12. Официальная документация Git : веб-сайт. — URL: <https://git-scm.com/doc> (дата обращения: 31.05.2026).